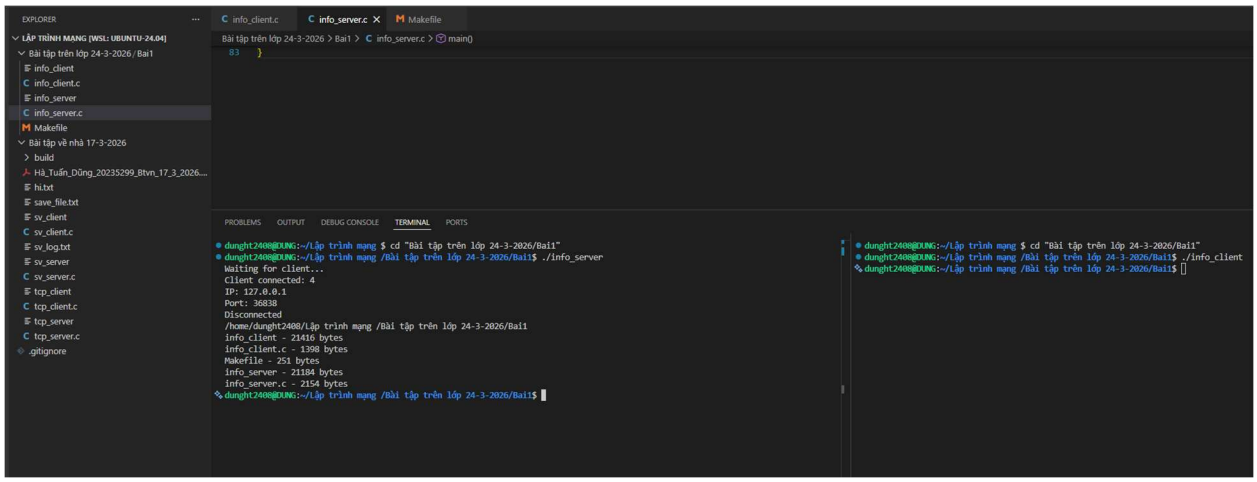
Báo cáo BTVN 24/3/2026

TCP Info Client-Server (Bài 1)

Mô tả cho Use case

Ứng dụng mô hình Client-Server qua TCP. Ngay khi kết nối, Client duyệt thư mục, bơm toàn bộ đường dẫn, tên file (ngăn cách bởi ký tự \0) và kích thước (raw bytes) thành một luồng dữ liệu liên tục. Ở chiều ngược lại, Server dùng recv() và realloc() gom toàn bộ payload này vào một bộ đệm duy nhất cho đến khi Client ngắt kết nối. Cuối cùng, Server giải mã bằng cách trượt con trỏ qua bộ đệm, dùng strcpy() bám theo mốc \0 để bóc tách chuỗi và memcpy() trích xuất kích thước, in toàn bộ kết quả ra màn hình.

Hình ảnh 1.1:



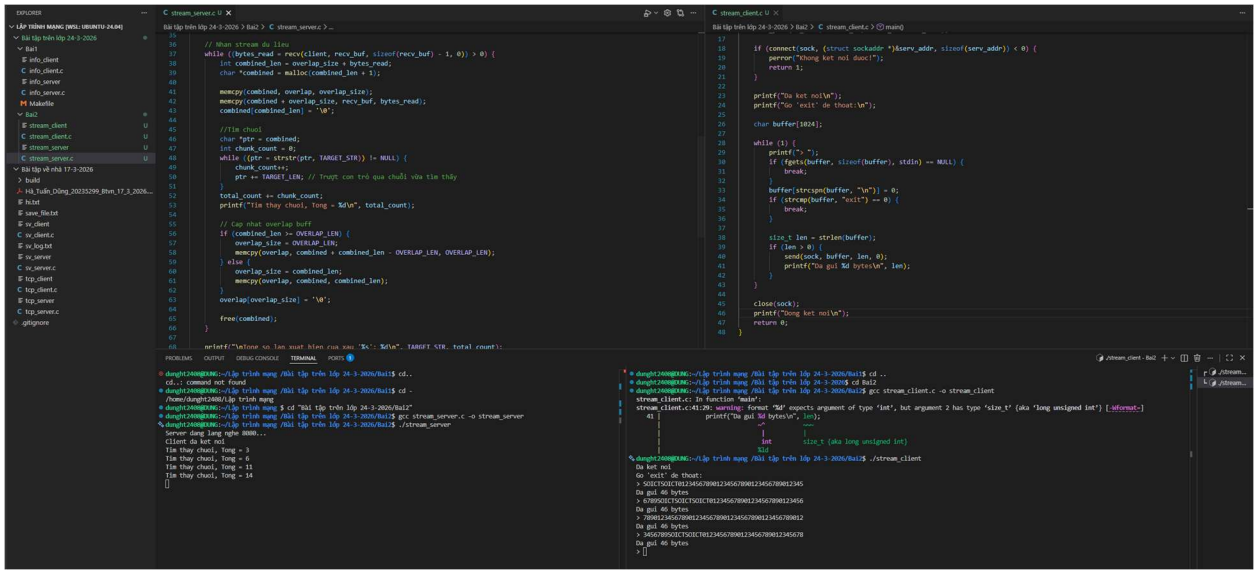
- **Phía Server (bên trái):** Hệ thống bắt được kết nối từ IP 127.0.0.1 qua cổng động 36838. Ngay khi nhận được tín hiệu ngắt từ Client, Server đã xử lý trọn vẹn bộ đệm. Kết quả in ra đúng chuỗi đường dẫn tuyệt đối (.../Bai1) và trích xuất thành công 5 tệp (gồm mã nguồn, file thực thi và Makefile) với dung lượng tính bằng byte hoàn toàn hợp lý.
- **Phía Client (bên phải):** Tiến trình đã hoàn thành tốt vòng đời: khởi tạo, quét thư mục, đóng gói payload đẩy xuống socket và thoát.

TCP Streaming (Bài 2)

Mô tả cho Use case

Ứng dụng mô hình Client-Server qua TCP để xử lý luồng dữ liệu văn bản. Client liên tục đọc dữ liệu nhập từ người dùng và đẩy qua socket. Ở chiều ngược lại, để tránh xâu cần tìm bị tách ở hai luồng khác nhau, Server tự động trích xuất 9 byte cuối của lần nhận trước để chắp nối vào ngay trước khối dữ liệu mới. Cuối cùng, hệ thống dùng hàm strstr() quét qua mảng tổng để đếm chính xác số lần xuất hiện của xâu "0123456789", đảm bảo không bỏ sót kết quả dù chuỗi bị cắt chéo giữa hai lần truyền.

Hình ảnh 2.1:



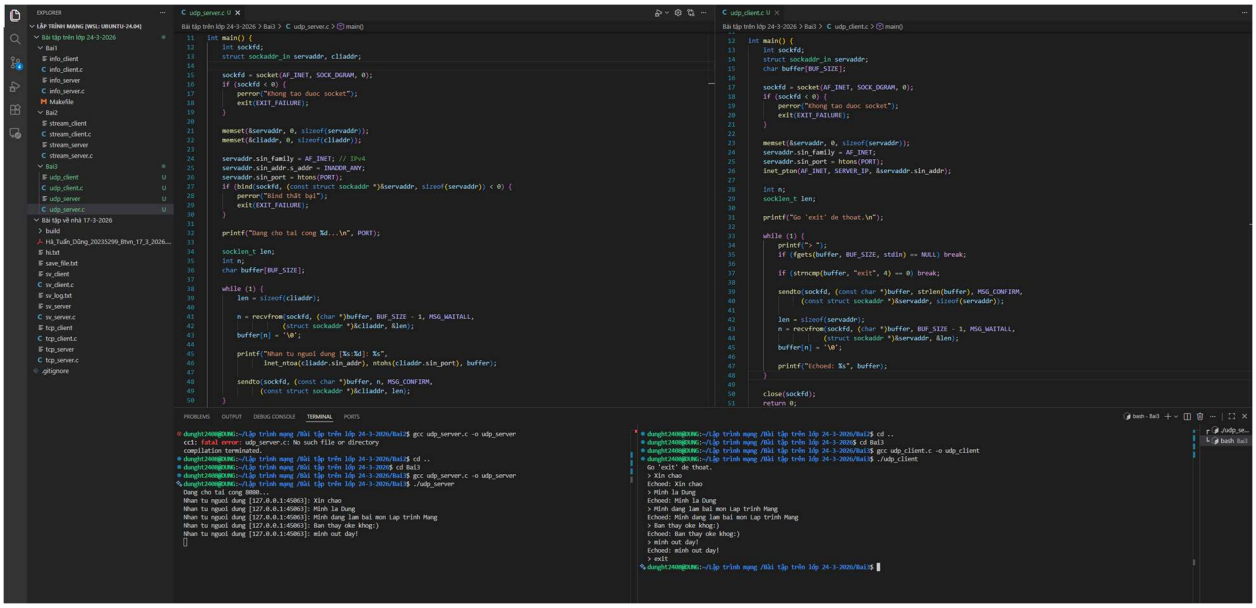
- **Phía Server (bên trái):** Hệ thống duy trì kết nối và xử lý byte-stream theo thời gian thực. Mỗi khi Client gửi một mảnh dữ liệu (kể cả khi chuỗi mục tiêu bị cắt đôi ngẫu nhiên), Server tiến hành ghép nối với bộ đệm gối đầu thành công, phát hiện và đếm chuẩn xác số lượng xâu "0123456789". Kết quả in ra màn hình thể hiện rõ số lượng byte nhận được từng đợt và tổng số chuỗi cộng dồn hoàn toàn chính xác.
- **Phía Client (bên phải):** Cho phép người dùng nhập các văn bản không liền mạch, được xử lí ký tự xuống dòng trước khi đẩy dữ liệu cho phía server. Khi kết thúc chỉ cần nhập exit để thoát.

UDP Echo (Bài 3)

Mô tả cho Use case

Ứng dụng mô hình Client-Server phi kết nối qua giao thức UDP. Khác với TCP, cả hai tiến trình sử dụng socket dạng SOCK_DGRAM và không thiết lập kênh truyền ảo. Client đọc dữ liệu từ người dùng, dùng hàm sendto() đóng gói thành các datagram độc lập và ném trực tiếp đến địa chỉ IP/Port của Server. Ở chiều ngược lại, Server dùng hàm recvfrom() để bắt gói tin, đồng thời tự động trích xuất cấu trúc địa chỉ nguồn (sockaddr_in) của Client. Ngay sau đó, Server sử dụng chính cấu trúc địa chỉ này để dội ngược (echo) nguyên bản payload về lại Client thông qua lệnh sendto().

Hình ảnh 2.1:



- **Phía Server (bên trái):** Mỗi khi recvfrom() bắt được dữ liệu, hệ thống phân giải và in ra chuẩn xác IP cùng Port động của Client. Việc lập tức dội ngược payload qua sendto() mà không cần duy trì trạng thái kết nối đã minh chứng rõ rệt cho bản chất phi kết nối và tốc độ phản hồi cực nhanh của socket UDP.
- **Phía Client (bên phải):** Các datagram chứa chuỗi ký tự được ném thẳng tới đích. Cặp hàm sendto() và recvfrom() phối hợp hoàn hảo: đẩy dữ liệu đi và nhận lại chuỗi phản hồi tức thì từ Server. Khi người dùng nhập 'exit', vòng lặp bị bẻ gãy, tiến trình gọi close() để giải phóng socket và thoát.